

Corso L^AT_EX 2026 - Elaborato

Luca Quaresima
VR500114

Indice

1	Introduzione	2
1.1	Programmazione Tradizionale	2
1.2	I Limiti dell'Approccio Tradizionale	2
1.3	Cos'è il Machine Learning	2
2	Manipolazione dei Dati	4
2.1	Introduzione ai Tensori	4
2.2	Primi passi con PyTorch	5

1. Introduzione

1.1 Programmazione Tradizionale

Fino a poco tempo fa, i programmi informatici seguivano regole rigide e predefinite. Ad esempio, nello sviluppo di un e-commerce, ogni azione dell'utente (come aggiungere un articolo al carrello) corrisponde a una precisa regola codificata a mano dai programmatori. Se un problema può essere risolto interamente con questo approccio deterministico, non c'è alcun bisogno di scomodare l'apprendimento automatico.

1.2 I Limiti dell'Approccio Tradizionale

Molti compiti complessi, tuttavia, non si piegano facilmente all'ingegno umano. Risulta infatti quasi impossibile scrivere da zero, basandosi solo su regole fisse, un programma in grado di:

- Prevedere il meteo analizzando dati satellitari e storici;
- Comprendere e rispondere a domande scritte in linguaggio libero;
- Identificare e contornare le persone presenti in una fotografia;
- Suggestire agli utenti prodotti mirati che non sapevano di desiderare.

1.3 Cos'è il Machine Learning

Il **Machine Learning** è lo studio di algoritmi capaci di imparare dall'esperienza. A differenza dei software tradizionali che restano statici finché un umano non li aggiorna, questi modelli migliorano le proprie prestazioni man mano che accumulano nuovi dati. Una sua branca fondamentale è il *Deep Learning*, che oggi guida le maggiori innovazioni in campi che vanno dalla computer vision fino alla genomica.

Caratteristica	Programmazione Classica	Machine Learning
Dati in ingresso	Dati e regole logiche	Dati ed etichette (esempi)
Risultato	Risposte o decisioni	Modello predittivo
Aggiornamento	Riscrittura del codice sorgente	Riaddestramento sui nuovi dati
Complessità	Limitata alla logica umana	Scalabile su problemi complessi

Tabella 1.1: Differenze strutturali tra approccio classico e apprendimento automatico.

2. Manipolazione dei Dati

2.1 Introduzione ai Tensori

Per poter ottenere dei risultati, abbiamo bisogno di un modo per memorizzare e manipolare i dati. Iniziamo a sporcarci le mani con gli array n -dimensionali, che chiameremo anche **tensori**. Per tutti i moderni framework di deep learning, la classe tensore assomiglia agli `ndarray` di NumPy, ma con due caratteristiche fondamentali in più:

- Supportano la differenziazione automatica.
- Sfruttano le GPU per accelerare il calcolo.

Tra i framework più popolari troviamo:

1. PyTorch
2. Tensorflow
3. JAX
4. MXNet

Framework	Sviluppatore	Linguaggio Principale
PyTorch	Meta AI	Python / C++
TensorFlow	Google	Python / C++
JAX	Google	Python

Tabella 2.1: Confronto tra i principali framework per tensori.

2.2 Primi passi con PyTorch

Un tensore rappresenta un array (possibilmente multidimensionale) di valori numerici. Nel caso unidimensionale è chiamato *vettore*, con due assi è chiamato *matrice*. Con k assi, ci riferiamo all'oggetto semplicemente come un tensore di ordine k .

Iniziamo importando la libreria. Tramite la funzione `arange(n)`, possiamo creare un vettore di valori equidistanti:

```
1 import torch
2 x = torch.arange(12, dtype=torch.float32)
3 print(x)
```

Possiamo cambiare la forma (shape) di un tensore senza alterarne la dimensione o i valori usando `reshape`. Da un punto di vista matematico, se consideriamo un tensore multidimensionale di dimensioni $d_1 \times d_2 \times \dots \times d_k$, il numero totale di elementi N è dato dal prodotto delle singole dimensioni. Quando eseguiamo un'operazione di rimodellamento (`reshape`) in una nuova forma a m dimensioni $(R_1, R_2, \dots, R_m)$, la cardinalità complessiva deve rimanere invariata:

$$N = \prod_{i=1}^k d_i = \prod_{j=1}^m R_j \quad (2.1)$$

Nel caso specifico di una trasformazione da vettore a matrice bidimensionale di forma (R, C) , l'equazione si semplifica e ci permette di ricavare il vincolo strutturale:

$$N = R \times C \implies C = \frac{N}{R} \quad (2.2)$$

Ecco come appare l'applicazione pratica di questo concetto tramite codice:

```
1 X = x.reshape(3, 4)
2 print(X)
```

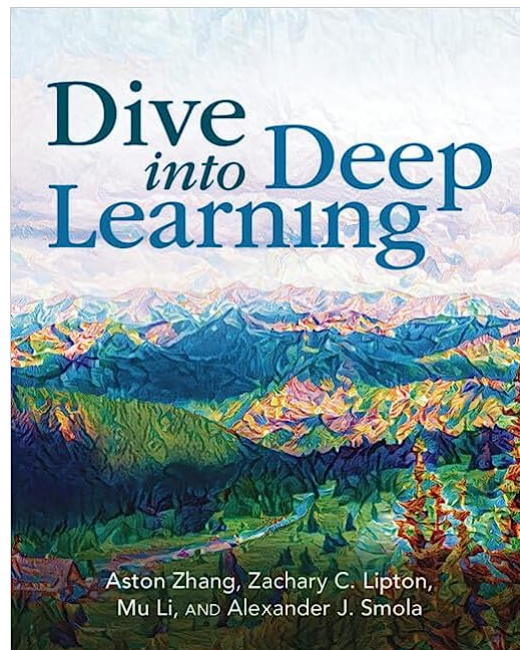


Figura 2.1: Testo di riferimento.